SUBJECT CODE: CSA5802
SUBJECT NAME: DEVSECOPS ENGINEERING AND APPLICATION SECURITY
FACULTY NAME: Dr. R. Pitchandi

Assignment Problem / Challenge

A rapidly growing digital financial services company operates a customer-facing web application, APIs and containerized backend services using Git-based development and automated deployment. A recent security review identified insecure code, vulnerable third-party dependencies, improper secret handling, excessive access privileges, insecure container or infrastructure configurations, fragmented logging and weak incident-response evidence. The company wants to improve security throughout its software delivery process without significantly reducing release speed.

Assume that you are the Senior DevSecOps Engineer responsible for securing this environment. Develop or select a functional web/API application and maintain it in a version-controlled repository. Analyze the application and delivery workflow, identify significant security risks, and redesign the process so that security is integrated from development through production. Secure coding, code review, secret handling and automated early security checks must be incorporated so that developers receive useful security feedback before deployment.

Implement a working automated CI/CD pipeline that builds and tests the application and performs security analysis of source code, third-party dependencies and the running application. Introduce controlled vulnerabilities in the laboratory project and demonstrate their detection. Configure security criteria that can block an unsafe release, show at least one failed pipeline execution, remediate the identified issue and re-run the pipeline successfully. Preserve the relevant scan results, pipeline logs and remediation history as evidence.

Containerize and deploy the application using a reproducible declarative configuration in a Docker, cloud or Kubernetes-based laboratory environment. Sensitive information must not be hard-coded in source control, and access permissions must follow least privilege. Validate the container and infrastructure/deployment configuration for security weaknesses and remediate significant findings. The deployment should demonstrate appropriate handling of identities, secrets and production configuration, together with a controlled resilience or malformed-input test where appropriate.

Establish centralized security logging and monitoring for the application and at least one additional operational layer. Create meaningful detection conditions and alert routing for suspicious activity. Generate a controlled security event and demonstrate that it can be traced from the original log to an alert. Then conduct one authorized incident

simulation in the laboratory, detect it through the implemented monitoring system, reconstruct the incident timeline, identify the root cause and affected assets, preserve relevant forensic evidence, and perform containment, eradication and recovery.

After the incident, implement a permanent security improvement and verify that the same weakness is prevented or detected more effectively. Use repository history, pipeline reports, deployment configuration, access-control records, monitoring data and incident evidence to demonstrate that the environment is auditable. Measure suitable security and delivery indicators and briefly evaluate the impact of the implemented controls on security risk, deployment speed, reliability, developer effort and operational cost.

The final submission must represent one integrated working solution rather than separate theoretical answers. It should include the complete project source, CI/CD configuration, container and deployment files, architecture/workflow diagrams, security scan evidence, failed and successful pipeline executions, monitoring and alert evidence, incident investigation records, remediation evidence, reproducible setup instructions and a concise technical report. Students must be prepared to demonstrate the system and justify their engineering decisions during evaluation. All vulnerability testing and incident simulation must be restricted to systems owned by the student/team or intentionally created laboratory targets.

Expected Outcome

The evaluator should be able to observe a complete lifecycle in which an application change enters source control, undergoes automated security validation, is blocked when an unacceptable weakness is detected, is remediated and successfully redeployed, and is continuously monitored. A simulated security incident should then be detected, investigated, contained and resolved using evidence generated by the implemented environment.

DEVSECOPS ENGINEERING AND APPLICATION SECURITY

Assignment Answer Report

Subject Code: CSA5802

Faculty Name: Dr. R. Pitchandi

Project Title: Secure DevSecOps Lifecycle for a Digital Financial Services Web/API Application

1. Introduction

This report presents an integrated DevSecOps solution for a rapidly growing digital financial services company. The laboratory application consists of a customer-facing web/API service with a containerized backend and Git-based development workflow. The solution integrates secure coding, code review, secret management, automated security testing, container and deployment security, centralized logging, monitoring, incident response, and continuous improvement.

The design follows the assignment requirement that security controls should be integrated from development through production without unnecessarily slowing release activities. The final lifecycle is designed so that an application change is committed to source control, automatically tested and scanned, blocked when an unacceptable security weakness is found, remediated, redeployed, and continuously monitored.

2. Problem Identification and Security Risks

The security review identified the following major risks in the original software delivery process:

Risk	Example	Impact	Control
Insecure code	Unsanitized input and weak validation	Injection, data exposure	Secure coding + SAST
Vulnerable dependencies	Outdated third-party package	Known CVE exploitation	SCA + dependency updates
Improper secret handling	API/database secret in source code	Credential compromise	Secret manager + secret scanning
Excessive privileges	Broad container/service permissions	Lateral movement	Least privilege + RBAC
Insecure containers	Untrusted image or root execution	Container compromise	Image scanning + hardened Dockerfile
Weak infrastructure	Unsafe deployment configuration	Unauthorized access	IaC scanning + policy checks
Fragmented logging	Logs stored separately	Poor investigation	Centralized logging + SIEM
Weak incident evidence	No consistent timeline or retention	Difficult forensics	Alerting + evidence preservation

3. Proposed Application and Technology Stack

A small financial transaction REST API is selected as the laboratory application. It provides controlled endpoints for authentication, customer records, and transaction processing. The application is intentionally designed to contain selected laboratory vulnerabilities so that the security pipeline can demonstrate detection and blocking.

- Source control: Git and a Git-based repository.
- Application: REST web/API service with a containerized backend.
- CI/CD: GitHub Actions or an equivalent Git-based CI/CD platform.
- Source-code security: Semgrep or equivalent SAST tool.
- Dependency security: OWASP Dependency-Check or equivalent SCA tool.
- Secret detection: Gitleaks or equivalent secret scanner.
- Container security: Trivy or equivalent image scanner.
- Dynamic testing: OWASP ZAP against the running laboratory application.
- Deployment: Docker Compose for a reproducible laboratory deployment.
- Monitoring: centralized application and host/container logs with alert rules.
- Incident response: documented detection, investigation, containment, eradication and recovery workflow.

4. Secure DevSecOps Architecture

The proposed architecture connects development, security validation, deployment and monitoring into one lifecycle. Developers commit application changes to the Git repository. The CI/CD pipeline automatically performs unit testing and security checks. Only a build that satisfies the security gate proceeds to deployment. The deployed application sends logs to centralized monitoring, where suspicious activity generates an alert for investigation.

Architecture flow:

Developer → Git Repository → CI Pipeline → Build & Unit Test → SAST → Secret Scan → Dependency Scan → Container Build → Container Scan → DAST → Security Gate → Deployment → Centralized Logging → Detection/Alerting → Incident Response → Remediation → Git Repository

5. Secure Coding and Code Review

Secure coding practices are incorporated before deployment. Input received through API parameters is validated for type, length and expected format. Database access uses parameterized queries. Authentication credentials and application secrets are never placed directly in source code. Error messages are designed not to expose internal implementation details.

- Use input validation and output encoding.
- Use parameterized database queries.

- Apply authentication and authorization checks to protected endpoints.
- Do not commit passwords, API keys or tokens.
- Use approved third-party packages and maintain current versions.
- Review security-sensitive changes through pull requests.
- Require automated security checks before merging protected branches.

6. CI/CD Security Pipeline

The pipeline is designed to provide fast feedback at early stages and deeper security validation before deployment.

Stage	Activity	Release Decision
Checkout	Retrieve version-controlled source	Continue
Unit Test	Run functional tests	Fail if tests fail
SAST	Analyze source code for insecure patterns	Fail on configured high-risk findings
Secret Scan	Detect hard-coded credentials and tokens	Block exposed secrets
SCA	Check third-party dependencies for known vulnerabilities	Block unacceptable severity
Build	Create application/container artifact	Fail on build error
Image Scan	Check container packages and configuration	Block critical findings
DAST	Test running application for web/API weaknesses	Block configured high-risk findings
Security Gate	Evaluate all security criteria	Deploy only if policy passes
Deploy	Deploy reproducible configuration	Continue
Post-deployment tests	Health and security checks	Rollback/fail if unsafe

7. Controlled Vulnerability Demonstration

For the laboratory demonstration, controlled vulnerabilities are introduced only into the intentionally created test application. No real third-party or unauthorized system is tested.

- Hard-coded laboratory API key: detected by secret scanning.
- Known vulnerable dependency version: detected by software composition analysis.
- Unsafe input handling: detected by SAST and/or DAST.

- Over-privileged container configuration: identified during container/deployment validation.

The pipeline security gate is configured so that an unacceptable finding causes the pipeline to fail. The vulnerable change is then removed or corrected, the dependency is upgraded, the insecure configuration is hardened, and the pipeline is executed again. The successful run becomes the evidence that the remediation satisfies the release policy.

8. Containerization and Deployment Security

The application is packaged as a reproducible container. The container configuration follows least privilege and avoids embedding sensitive values in the image or source repository.

- Use a minimal trusted base image.
- Run the application as a non-root user.
- Expose only required ports.
- Keep secrets outside source control and inject them at runtime.
- Pin or control important dependency and image versions.
- Use read-only filesystems or dropped Linux capabilities where practical.
- Scan the image before deployment.
- Use separate configuration for development and production-like laboratory environments.

9. Identity, Secrets and Least Privilege

Sensitive information is stored outside source control and supplied to the application through protected CI/CD secrets or a secret-management mechanism. Repository and deployment permissions follow least privilege. Developers, CI runners, application services and monitoring components receive only the permissions required for their functions.

Access records are retained as audit evidence. Repository branch protection and pull-request review reduce the possibility of unauthorized changes reaching production-like deployment.

10. Resilience and Malformed-Input Test

A controlled malformed-input test is performed against the laboratory API. Examples include invalid data types, excessive input length and unexpected request parameters. The expected result is graceful rejection with an appropriate error response, without exposing stack traces or sensitive information. The application health check is also used to confirm that the service remains available after the test.

11. Centralized Security Logging and Monitoring

Application logs and at least one operational layer, such as container or host logs, are collected centrally. The monitoring system creates detection conditions for suspicious activity.

Log Source	Detection Condition	Alert
API authentication log	Repeated failed authentication attempts	Authentication abuse alert
Web/API access log	Suspicious input or repeated error responses	Possible attack alert
Container log	Unexpected process or repeated application failure	Container security alert
Access/audit log	Unexpected privileged operation	Privilege misuse alert

A controlled security event is generated in the laboratory. The event is traced from the original application/container log to the detection rule and then to the generated alert. The event timestamp, source, affected asset and response action are recorded.

12. Incident Simulation and Investigation

One authorized laboratory incident is simulated to validate the monitoring and response process. The investigation reconstructs the timeline from centralized logs and CI/CD records.

Incident Response Phase	Action
Detection	Monitoring rule identifies the controlled suspicious event.
Analysis	Correlate application, container and access logs.
Timeline	Record first event, escalation, detection and response times.
Root Cause	Identify the insecure change, configuration or credential exposure responsible.
Affected Assets	Identify application endpoint, container, repository/deployment artifact and relevant account.
Evidence Preservation	Preserve logs, pipeline output, scan reports and repository history.
Containment	Restrict the affected service, credential or access path.
Eradication	Remove the vulnerability, malicious/unsafe change or exposed

	secret.
Recovery	Redeploy the corrected artifact and validate service health.
Lessons Learned	Add a permanent control to prevent or detect recurrence.

13. Permanent Security Improvement

After the simulated incident, the identified weakness is permanently addressed. For example, if a secret was exposed in source code, the secret is revoked and rotated, secret scanning is made mandatory, and the secret is moved to protected runtime configuration. If an insecure input-handling weakness was detected, validation is strengthened and a regression test is added.

The same test is then repeated. The expected result is that the weakness is either blocked earlier in the CI/CD pipeline or detected more effectively during deployment monitoring.

14. Auditability and Evidence

The environment is designed to remain auditable through retained technical evidence. The following evidence should be preserved with the final submission:

- Git repository history showing application changes and remediation.
- CI/CD configuration file and pipeline execution history.
- At least one failed security pipeline execution.
- Successful pipeline execution after remediation.
- SAST, secret, dependency, container and DAST scan reports.
- Container and deployment configuration files.
- Access-control and permission records.
- Centralized log records and alert evidence.
- Incident timeline and investigation record.
- Containment, eradication and recovery evidence.
- Final verification showing the weakness is prevented or detected.

15. Security and Delivery Metrics

Metric	Purpose	Expected Evaluation
Pipeline security pass rate	Shows release quality	Track failed vs successful runs
Mean time to detect	Measures monitoring effectiveness	Compare event time with alert time
Mean time to remediate	Measures response efficiency	Track from finding to verified fix

Critical/high findings per release	Measures security exposure	Target reduction over time
Secret leakage count	Measures secret-handling quality	Target zero
Deployment duration	Measures delivery impact	Compare before and after controls
Rollback/recovery time	Measures reliability	Track time to restore service
Developer remediation effort	Measures engineering cost	Record effort required per finding

16. Evaluation of the Proposed Controls

The integrated solution improves security by moving detection earlier in the software lifecycle. Automated checks provide developers with feedback before deployment, while the release gate prevents unacceptable weaknesses from reaching the deployed environment. Container and infrastructure validation reduce configuration risk, and centralized monitoring improves the ability to detect and investigate suspicious activity.

The main delivery impact is additional automated pipeline stages and the effort required to remediate findings. Because the checks are automated and ordered from fast checks to deeper checks, the process can preserve release speed while improving security. Reusable pipeline definitions, container images and test cases reduce repeated developer effort over time.

17. Reproducible Setup Procedure

1. Create the Git repository and add the web/API application source.
2. Add the Dockerfile and deployment configuration.
3. Configure protected CI/CD secrets without committing sensitive values.
4. Add unit tests and security scanning stages to the CI/CD workflow.
5. Configure severity thresholds for the security gate.
6. Run the pipeline with the controlled vulnerable version and preserve the failed result.
7. Remediate the detected vulnerability and commit the fix.
8. Re-run the pipeline and preserve the successful result.
9. Deploy the corrected containerized application.
10. Configure centralized application and operational logging.
11. Create detection rules and alert routing.
12. Generate the authorized laboratory security event.
13. Investigate the event and preserve the evidence.

14. Perform containment, eradication and recovery.

15. Implement the permanent security improvement and verify the same weakness is prevented or detected.

18. Conclusion

The proposed DevSecOps solution provides one integrated lifecycle for a digital financial services application. Security is incorporated into development, source control, CI/CD, containerization, deployment and production-like monitoring. The design satisfies the assignment objective of demonstrating a vulnerable laboratory change, an automatically blocked release, remediation, successful redeployment, centralized detection, an authorized incident simulation, evidence preservation and permanent security improvement.

All vulnerability testing and incident simulation must remain restricted to systems owned by the student/team or intentionally created laboratory targets. The final demonstration should use actual repository history, pipeline results, scan reports, logs, alerts and incident records rather than fabricated evidence.

Appendix A – Evidence Checklist

Evidence Item	Status to Demonstrate	Attachment/Reference
Architecture/workflow diagram	Required	Attach final diagram
Repository history	Required	Attach screenshot/export
Failed CI/CD execution	Required	Attach screenshot/log
Successful CI/CD execution	Required	Attach screenshot/log
SAST result	Required	Attach scan report
Secret scan result	Required	Attach scan report
Dependency scan result	Required	Attach scan report
Container scan result	Required	Attach scan report
DAST result	Required	Attach scan report
Deployment configuration	Required	Attach Docker/Compose/IaC files
Centralized log evidence	Required	Attach screenshot/export
Alert evidence	Required	Attach screenshot/export
Incident timeline	Required	Attach investigation record
Remediation verification	Required	Attach final scan/pipeline result

Appendix B – Note on Implementation Evidence

This report is prepared from the CSA5802 assignment problem statement. The assignment requires a working implementation and real evidence such as failed/successful pipeline executions, scan results, monitoring alerts and incident records. Those artifacts cannot be truthfully generated from the question sheet alone. They should be added after the laboratory implementation is executed.